

C05-AT3

MLA0201-Fundamentals Of Machine Learning

Mock Interview – Model Answers

Q1. Using Bayesian Networks to Predict Machine Failures in an Industrial Monitoring System

A Bayesian Network (BN) is a directed acyclic graph in which nodes represent random variables — such as sensor readings, vibration levels, temperature, pressure, and operating hours — and edges represent probabilistic dependencies between them. For an industrial monitoring system, I would model each machine component as a node, with parent nodes representing root causes (e.g., wear and tear, overheating) and child nodes representing observable symptoms (e.g., abnormal vibration, unusual noise) and the final failure event.

Approach

- Identify variables: sensor signals, environmental conditions, maintenance history, and failure status.
- Construct the graph structure using domain expertise or structure-learning algorithms, ensuring causal dependencies flow from root causes to observed symptoms to failure.
- Define Conditional Probability Tables (CPTs) for each node using historical failure logs and sensor data.
- Use exact inference (variable elimination) or approximate inference (sampling) to compute the posterior probability of failure given current sensor evidence.
- Continuously update the network with new evidence to get real-time failure probability estimates.

This allows the system to reason under uncertainty — for example, if vibration is high but temperature is normal, the network can still estimate the likelihood of bearing failure versus misalignment, enabling predictive maintenance before a breakdown occurs.

Key Points to Remember

- BN = directed acyclic graph capturing cause-effect relationships under uncertainty.
- Nodes = sensor variables and failure states; edges = probabilistic dependencies.
- CPTs learned from historical data drive posterior probability of failure.
- Enables real-time predictive maintenance through evidence-based inference.

Q2. Using Markov Random Fields to Model Neighborhood Pixel Dependencies for Image Denoising

A Markov Random Field (MRF) is an undirected graphical model well-suited to image processing because pixel intensities are spatially correlated — a pixel's true value is strongly influenced by its immediate neighbors. In an MRF formulation for denoising, each pixel is treated as a node, and edges connect neighboring pixels (typically a 4- or 8-connected neighborhood), enforcing the Markov property that a pixel depends only on its neighbors, not on distant pixels directly.

Approach

- Model the joint distribution of the image using an energy function with two terms: a data term (penalizing deviation between the observed noisy pixel and the estimated true pixel) and a smoothness/prior term (penalizing large intensity differences between neighboring pixels).
- Express this as a Gibbs distribution, since MRFs and Gibbs distributions are equivalent under the Hammersley-Clifford theorem.
- Minimize the total energy using techniques such as Iterated Conditional Modes (ICM), Belief Propagation, or Graph Cuts to find the most probable denoised image.
- The smoothness term encourages neighboring pixels to have similar values, effectively suppressing random noise while preserving edges if an edge-preserving prior (e.g., Total Variation) is used.

This local-dependency structure is exactly why MRFs are effective for denoising: they exploit spatial correlation without requiring a full joint model of every pixel in the image.

Key Points to Remember

- MRF = undirected graph capturing local spatial dependencies between pixels.
- Energy function = data term + smoothness (neighborhood) term.
- Equivalent to a Gibbs distribution (Hammersley-Clifford theorem).
- Optimized via ICM, Belief Propagation, or Graph Cuts to denoise while preserving edges.

Q3. Applying a Hidden Markov Model to Process Sequential Audio Signals in Speech Recognition

Speech is inherently sequential — phonemes and words unfold over time, and the acoustic signal at any moment depends on an underlying, unobserved linguistic state. A Hidden Markov Model (HMM) is a natural fit because it models a sequence of hidden states (e.g., phonemes or sub-phoneme units) that are not directly observed, along with observable emissions (e.g., MFCC feature vectors extracted from the audio) generated from those states.

Approach

- Define hidden states representing phonemes or acoustic sub-units, with a transition probability matrix capturing how likely one phoneme is to follow another.
- Define emission probabilities (often modeled with Gaussian Mixture Models or, in modern systems, neural network outputs) that describe how likely a given audio feature vector is to be produced by each hidden state.
- Use the Forward-Backward algorithm to compute the likelihood of an observed audio sequence given the model.
- Use the Viterbi algorithm to decode the most probable sequence of hidden states (phonemes/words) corresponding to the observed audio.
- Train the transition and emission probabilities using the Baum-Welch (Expectation-Maximization) algorithm on labeled or semi-labeled speech corpora.

This allows the recognizer to convert a continuous, noisy audio waveform into the most likely sequence of words by combining acoustic evidence with learned temporal transition patterns.

Key Points to Remember

- HMM models hidden linguistic states generating observable audio features.
- Transition matrix = phoneme-to-phoneme sequencing; emissions = acoustic feature likelihoods.
- Forward-Backward computes sequence likelihood; Viterbi decodes the best state path.
- Baum-Welch (EM) trains model parameters from speech data.

Q4. Explaining Conditional Random Fields for Sequence Labeling Tasks such as POS Tagging

A Conditional Random Field (CRF) is an undirected, discriminative graphical model used for structured prediction, where the goal is to predict a sequence of labels (e.g., part-of-speech tags) for a sequence of observations (e.g., words in a sentence). Unlike generative models such as HMMs, a CRF directly models the conditional probability of the label sequence given the entire observation sequence, $P(Y|X)$, rather than the joint probability $P(X,Y)$.

- Represent the sentence as a linear-chain graph where each word corresponds to a label node, and edges connect adjacent labels to capture sequential dependencies (e.g., a determiner is often followed by a noun or adjective).
- Define feature functions that can use rich, overlapping evidence from the entire input sequence — such as the word itself, its prefix/suffix, capitalization, surrounding words, and even the whole sentence — not just the current observation as HMMs assume.
- Combine feature functions with learned weights to compute a global score for a candidate label sequence, normalized over all possible sequences using a partition function.
- Use the Viterbi algorithm for efficient decoding of the most probable label sequence, and train weights via maximum likelihood with gradient-based optimization.

Because CRFs condition on the entire observation sequence and avoid the independence assumptions HMMs make about observations, they typically achieve higher tagging accuracy in practice.

Key Points to Remember
● CRF is a discriminative, undirected model predicting $P(Y X)$ directly. ● Linear-chain structure captures dependencies between adjacent labels. ● Rich, overlapping feature functions can use the full input context. ● Viterbi decoding + gradient-based training; generally outperforms HMMs for tagging.

Q5. How Conditional Independence Simplifies Probabilistic Inference – A Graphical Model Example

Conditional independence is the property that two variables become independent of each other once a third variable (or set of variables) is known. Graphical models exploit this property to factorize a complex joint probability distribution into a product of smaller, local terms, which drastically reduces the number of parameters needed and makes inference computationally tractable.

Example

Consider a simple Bayesian Network: $\text{Rain} \rightarrow \text{Sprinkler} \rightarrow \text{Wet Grass}$, along with $\text{Rain} \rightarrow \text{Wet Grass}$ directly. Without any independence assumptions, the joint distribution over all variables would require an exponential number of parameters as more variables are added. However, the graph structure tells us that Wet Grass is conditionally independent of Rain given the states of both Sprinkler and the direct causal path — more generally, each node is conditionally independent of its non-descendants given its parents (the local Markov property).

Approach

- Use the graph's d-separation criterion to determine which variables are conditionally independent given evidence.
- Factorize the joint distribution as a product of each variable's conditional probability given only its parents, instead of a full joint table.
- Perform inference (e.g., variable elimination) by summing out irrelevant variables early, using the independence structure to avoid recomputation over the full joint space.

This factorization is precisely why graphical models scale to problems with many variables — conditional independence turns an intractable joint distribution into a manageable product of local, low-dimensional factors.

Key Points to Remember

- Conditional independence: two variables are independent given a third.
- Graphs factorize the joint distribution into local conditional terms.
- d-separation identifies independence relations directly from graph structure.
- Reduces parameters and computation, making inference tractable at scale.

Q6. Comparing Directed and Undirected Graphical Models for Symptom Relationships in Healthcare Diagnosis

In healthcare diagnosis, both directed graphical models (Bayesian Networks) and undirected graphical models (Markov Random Fields / Markov Networks) can represent relationships between diseases and symptoms, but they differ in what kind of relationships they naturally express.

Directed Graphical Models (Bayesian Networks)

- Edges represent causal or generative relationships, e.g., Disease $\rightarrow$ Symptom, which matches the natural causal story of how a disease produces observable symptoms.
- Conditional Probability Tables are often interpretable by clinicians (e.g., probability of fever given flu).
- Well suited when the causal direction between variables is known or clinically meaningful.

Undirected Graphical Models (Markov Networks)

- Edges represent symmetric correlations or co-occurrence without implying causal direction, useful when two symptoms tend to appear together but neither clearly causes the other.
- Better suited for modeling mutual influence between symptoms (e.g., symptom clusters) rather than a clear disease-to-symptom hierarchy.
- Inference relies on potential functions and partition functions rather than conditional probability tables.

In practice, I would use a Bayesian Network when the disease-to-symptom causal structure is clinically well established (making diagnosis interpretable), and a Markov Network when symptoms are highly interdependent without a clear causal hierarchy, such as correlated lab markers.

Key Points to Remember
• Bayesian Networks: directed, causal, interpretable disease-to-symptom relationships. • Markov Networks: undirected, symmetric correlations without implied causality. • Choice depends on whether causal direction between symptoms/diseases is known. • Both factorize the joint distribution but differ in local factor semantics (CPTs vs potentials).

Q7. Performing Bayesian Inference for Diagnosis When Patient Symptom Data Is Partially Missing

One of the key strengths of Bayesian Networks is that they naturally handle missing or partial evidence, since inference is performed over whatever variables are observed, while unobserved variables are simply marginalized out rather than treated as errors.

Approach

- Treat the available symptoms as observed evidence nodes and the missing symptoms as latent (unobserved) variables in the network.
- Use exact inference methods such as variable elimination or junction tree algorithms to marginalize over the missing variables, computing the posterior probability of each candidate disease given only the observed evidence.
- For larger networks where exact inference is expensive, use approximate inference such as Gibbs sampling or Markov Chain Monte Carlo (MCMC) to estimate the posterior distribution.
- Optionally, use the model itself to estimate the most probable values of the missing symptoms (via Maximum A Posteriori inference), which can support clinicians in deciding which additional tests to prioritize.

Because the joint distribution is factorized according to the graph structure, missing values do not break the model — the posterior over the diagnosis is simply computed by integrating over the uncertainty in the unobserved symptoms, giving a well-calibrated probability estimate rather than a brittle prediction.

Key Points to Remember

- Missing symptoms are treated as latent variables, not as errors.
- Variable elimination / junction tree marginalizes over missing evidence exactly.
- MCMC or Gibbs sampling used when exact inference is too costly.
- Posterior probability remains well-calibrated despite incomplete evidence.

Q8. Using a Hidden Markov Model to Analyze Customer Purchase Sequences in E-Commerce Recommendations

Customer purchasing behavior often follows latent stages — such as browsing, comparison-shopping, and ready-to-buy — that are not directly observable but influence the sequence of actions a customer takes (views, cart additions, purchases). A Hidden Markov Model is well suited to capture this because it models a sequence of hidden behavioral states that generate observable events over time.

Approach

- Define hidden states representing customer intent stages (e.g., exploring, interested, ready-to-purchase, churn-risk).
- Define observable emissions as the customer's recorded actions — product views, category switches, cart additions, purchases, or session gaps.
- Learn transition probabilities describing how customers typically move between intent stages, and emission probabilities describing which actions are likely in each stage, using the Baum-Welch algorithm on historical clickstream and purchase data.
- Use the Forward algorithm to continuously estimate the probability that a customer is currently in a 'ready-to-purchase' state, and use the Viterbi algorithm to infer the most likely path of intent states a customer has followed.
- Feed the inferred current state into the recommendation engine to serve stage-appropriate suggestions — e.g., complementary products for a ready-to-buy customer, or re-engagement offers for a churn-risk customer.

Key Points to Remember

- Hidden states = latent customer intent stages (browse, interested, ready-to-buy).
- Observations = clickstream/purchase events generated from those hidden states.
- Baum-Welch trains transition/emission probabilities from historical sequences.
- Forward/Viterbi infer current intent to drive stage-appropriate recommendations.

Q9. Why Conditional Random Fields Often Outperform Hidden Markov Models in Named Entity Recognition

Named Entity Recognition (NER) requires labeling each word in a sentence (e.g., as Person, Organization, Location, or Other) based on rich contextual cues. CRFs tend to outperform HMMs on this task for several structural reasons.

Key Reasons

- Discriminative vs generative modeling: CRFs model $P(Y|X)$ directly and optimize for label accuracy, whereas HMMs model the joint $P(X,Y)$ and must also expend capacity modeling how observations are generated, which is not needed for a labeling task.
- Feature richness: HMMs assume each observation depends only on the current hidden state, limiting them to simple, local features. CRFs can use arbitrary, overlapping features drawn from the entire input sequence — word shape, capitalization, prefixes/suffixes, gazetteer membership, neighboring words — without violating any independence assumption.
- No strong independence assumptions on observations: HMMs assume observation independence given the state, which is often violated in language data (e.g., words are correlated with surrounding words). CRFs make no such assumption on the input.
- Global normalization: CRFs normalize over the entire label sequence rather than making local per-state decisions, avoiding the 'label bias problem' that can affect some sequence models with per-state normalization.

In short, CRFs trade the generative simplicity of HMMs for a discriminative model that can exploit far richer context, which directly translates into higher precision and recall on NER benchmarks.

Key Points to Remember

- CRF is discriminative ($P(Y|X)$); HMM is generative ($P(X,Y)$).
- CRFs support rich, overlapping, whole-sequence features; HMMs are limited to local emissions.
- HMMs assume observation independence given state — often violated in text.
- CRFs use global sequence normalization, avoiding label bias issues.

Q10. How Graphical Model Learning and Generalization Enhance Fraud Detection Accuracy

Fraud detection involves identifying rare, evolving patterns across many correlated variables — transaction amount, location, device, time, merchant category, and account history. Graphical models are valuable here because they explicitly learn the dependency structure between these variables and generalize that structure to new, unseen transactions rather than relying purely on memorized examples.

Approach

- Structure learning: learn which variables are directly dependent on each other (e.g., transaction location and device are correlated, but both depend on account behavior), producing a graph that reflects real fraud patterns rather than assuming full independence or full dependence.
- Parameter learning: estimate conditional probabilities or potential functions from historical labeled fraud/non-fraud data, allowing the model to assign a calibrated probability of fraud to a new transaction rather than a hard binary decision.
- Regularization for generalization: apply sparsity-inducing priors (e.g., L1 regularization on structure or parameters) to avoid overfitting to noise in rare fraud cases, ensuring the model generalizes to new fraud patterns rather than memorizing historical ones.
- Inference under uncertainty: use the learned graph to reason about partially observed transactions in real time, combining weak individual signals (e.g., slightly unusual location plus slightly unusual amount) into a strong joint fraud probability that a rule-based system would miss.
- Continuous re-learning: periodically retrain the structure and parameters as fraud patterns evolve, allowing the model to adapt rather than remain static.

By explicitly modeling dependencies instead of treating features independently, graphical models capture the subtle, joint patterns that characterize fraud, while proper learning and regularization ensure the model generalizes well to new and evolving fraud tactics rather than overfitting to historical cases.

Key Points to Remember

- Structure learning captures true dependencies between transaction variables.
- Parameter learning gives calibrated fraud probabilities, not just hard labels.
- Regularization (e.g., sparsity) prevents overfitting to rare historical fraud cases.
- Joint reasoning over correlated weak signals improves detection of subtle fraud patterns.